

Comprehensive ANN Cheat Sheet

Neural Network Fundamentals

Basic Components

- **Perceptron:** Single neuron that maps inputs to output via weights, bias, and activation
- **Forward propagation:** Input $\rightarrow$ weighted sum $\rightarrow$ activation $\rightarrow$ output
- **Backward propagation:** Calculate gradients with chain rule and update weights
- **Layer:** Collection of neurons processing information in parallel
- **Weights:** Learnable parameters that define strength of connections between neurons
- **Bias:** Learnable offset parameter for each neuron

Gradient Challenges

Problem: Vanishing/exploding gradients

Solutions:

- **ReLU:** $f(x) = \max(0, x)$ - Simple, fast, but "dying ReLU" problem
- **Leaky ReLU:** $f(x) = \max(\alpha x, x)$ where α is small - Prevents dying neurons
- **ELU:** Exponential for negative values - Smoother gradient near zero
- **Weight Initialization:** Proper initialization (He, Xavier) to maintain variance across layers
- **Gradient Clipping:** Limit gradient magnitude to prevent explosion
- **Residual Connections:** Allow gradients to flow more efficiently through the network

Activation Functions

For Hidden Layers

- **ReLU:** $f(x) = \max(0, x)$ - Most common, fast computation
- **Leaky ReLU:** $f(x) = \max(\alpha x, x)$ - Prevents dead neurons
- **ELU:** $f(x) = x$ if $x > 0$ else $\alpha(e^x - 1)$ - Smooth, negative values
- **SELU:** Self-normalizing variant of ELU
- **Swish:** $f(x) = x * \text{sigmoid}(x)$ - Smooth, non-monotonic
- **GELU:** $f(x) = x * \Phi(x)$ where Φ is the CDF of normal distribution
- **Mish:** $f(x) = x * \tanh(\text{softplus}(x))$ - Smooth, non-monotonic

For Output Layers

- **Binary Classification:** Sigmoid - outputs between 0-1
- **Multi-class Classification:** Softmax - converts logits to probabilities
- **Regression:** Linear (no bounds), ReLU (≥ 0), TanH (-1 to 1)

Loss Functions

Regression

- **MSE (Mean Squared Error):** Standard, punishes larger errors more severely
- **MAE (Mean Absolute Error):** Less sensitive to outliers, constant gradient
- **Huber:** Combines MSE and MAE benefits (parameter controlled)
- **Log-cosh:** Smooth approximation of Huber loss
- **Quantile Loss:** For predicting distribution quantiles

Classification

- **Binary Cross-Entropy:** For two-class problems
- **Categorical CE:** For multi-class with one-hot vectors
- **Sparse Categorical CE:** For multi-class with integer labels
- **Focal Loss:** Modified CE that focuses on hard examples
- **Hinge Loss:** Used in SVMs and margin-based classifiers
- **KL Divergence:** Measures difference between probability distributions

Optimizers

Basic Gradient Descent Variants

- **Gradient Descent:** Uses full dataset, stable but slow
- **Stochastic GD (SGD):** Single sample updates, faster but noisy
- **Mini-Batch GD:** Balance between full and stochastic GD

Advanced Optimizers

- **SGD with Momentum:** Adds velocity to reduce oscillations
- **Nesterov Accelerated Gradient:** Looks ahead to correct momentum
- **Adagrad:** Adapts learning rates per parameter (slows down over time)
- **RMSprop:** Fixes Adagrad's diminishing learning rates
- **Adadelta:** Eliminates need for a global learning rate
- **Adam:** Combines momentum and adaptive rates (generally best)
- **AdamW:** Adam with decoupled weight decay
- **Nadam:** Adam with Nesterov momentum
- **LAMB:** Helps training with large batch sizes
- **Lookahead:** Wraps other optimizers with outer optimization loop

Regularization Techniques

Weight Regularization

- **L1 Regularization:** Adds sum of absolute weights to loss (sparsity)
- **L2 Regularization:** Adds sum of squared weights to loss (weight decay)
- **Elastic Net:** Combines L1 and L2 regularization

Dropout Techniques

- **Dropout:** Randomly deactivates neurons during training
- **Spatial Dropout:** Drops entire feature maps in CNNs
- **DropConnect:** Randomly drops connections instead of neurons
- **Zoneout:** Special variant for RNNs

Normalization

- **Batch Normalization:** Normalizes activations by batch statistics
- **Layer Normalization:** Normalizes across features within each example
- **Weight Normalization:** Decouples magnitude and direction of weight vectors
- **Instance Normalization:** Like BN but normalizes each instance individually
- **Group Normalization:** Middle ground between Layer and Instance Norm

Other Regularization

- **Early Stopping:** Stop training when validation metrics plateau
- **Label Smoothing:** Softens one-hot encodings (e.g., 0.9 instead of 1.0)
- **Mixup:** Trains on weighted combinations of samples and targets
- **Data Augmentation:** Artificially creates more training examples

ANN Implementation

Sequential API (TensorFlow/Keras)

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.regularizers import l2

model = Sequential([
    Dense(128, input_shape=(input_dim,), activation='relu', kernel_regularizer=l2(0.001)),
    BatchNormalization(),
    Dropout(0.2),
    Dense(64, activation='relu', kernel_regularizer=l2(0.001)),
    BatchNormalization(),
    Dropout(0.2),
    Dense(output_dim, activation='sigmoid') # sigmoid/softmax/linear based
on task
])
```

```

model.compile(
    optimizer=Adam(learning_rate=0.001),
    loss='binary_crossentropy', # or categorical_crossentropy/mse
    metrics=['accuracy']
)

history = model.fit(
    X_train, y_train,
    epochs=20,
    batch_size=32,
    validation_split=0.2
)

```

Functional API

```

from tensorflow.keras.models import Model
from tensorflow.keras.layers import Input, Dense, Dropout, concatenate, BatchNormalization

# Multiple inputs example
input_A = Input(shape=(input_dim_A,))
input_B = Input(shape=(input_dim_B,))

# Branch A
x_A = Dense(64, activation='relu')(input_A)
x_A = BatchNormalization()(x_A)
x_A = Dropout(0.2)(x_A)

# Branch B
x_B = Dense(32, activation='relu')(input_B)
x_B = BatchNormalization()(x_B)
x_B = Dropout(0.2)(x_B)

# Merge branches
merged = concatenate([x_A, x_B])
output = Dense(output_dim, activation='softmax')(merged) # sigmoid/softmax/linear

```

```

model = Model(inputs=[input_A, input_B], outputs=output)

model.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

history = model.fit(
    [X_A_train, X_B_train],
    y_train,
    epochs=20,
    batch_size=32,
    validation_split=0.2
)

```

Custom Learning Rate Schedules

Step Decay

```

from tensorflow.keras.callbacks import LearningRateScheduler

def step_decay(epoch):
    initial_lr = 0.1
    drop_factor = 0.5
    epochs_drop = 10
    lr = initial_lr * (drop_factor ** (epoch // epochs_drop))
    return lr

lr_scheduler = LearningRateScheduler(step_decay)

```

Exponential Decay

```

def exp_decay(epoch):
    initial_lr = 0.1
    k = 0.1 # decay rate
    lr = initial_lr * np.exp(-k*epoch)

```

```
return lr
```

```
lr_scheduler = LearningRateScheduler(exp_decay)
```

Cosine Annealing

```
def cosine_annealing(epoch, total_epochs=100, eta_min=1e-6, eta_max=0.1):
```

```
    return eta_min + 0.5 * (eta_max - eta_min) * (1 + np.cos(np.pi * epoch / total_epochs))
```

```
lr_scheduler = LearningRateScheduler(lambda epoch: cosine_annealing(epoch))
```

ReduceLROnPlateau

```
from tensorflow.keras.callbacks import ReduceLROnPlateau
```

```
reduce_lr = ReduceLROnPlateau(  
    monitor='val_loss',  
    factor=0.2, # multiply lr by this factor  
    patience=5, # number of epochs with no improvement  
    min_lr=1e-6,  
    verbose=1  
)
```

Initialization Strategies

Common Initializers

- **Zeros:** All weights = 0 (generally only for biases)
- **Ones:** All weights = 1 (rarely used)
- **Random Normal:** Gaussian distribution with mean=0, std=0.01
- **Random Uniform:** Uniform distribution between -0.05 and 0.05
- **Xavier/Glorot:** Variance scaled by 1/n (good for tanh/sigmoid)

- **He Initialization:** Variance scaled by $2/n$ (good for ReLU)
- **LeCun:** Variance scaled by $1/n$

```
from tensorflow.keras.initializers import HeNormal, GlorotUniform

# Using initializers in layers
Dense(64, activation='relu', kernel_initializer=HeNormal())
Dense(64, activation='sigmoid', kernel_initializer=GlorotUniform())
```

Model Visualization and Evaluation

Plot Training History

```
import matplotlib.pyplot as plt

# Accuracy plot
plt.figure(figsize=(12, 4))
plt.subplot(1, 2, 1)
plt.plot(history.history['accuracy'])
plt.plot(history.history['val_accuracy'])
plt.title('Model Accuracy')
plt.ylabel('Accuracy')
plt.xlabel('Epoch')
plt.legend(['Train', 'Validation'], loc='lower right')

# Loss plot
plt.subplot(1, 2, 2)
plt.plot(history.history['loss'])
plt.plot(history.history['val_loss'])
plt.title('Model Loss')
plt.ylabel('Loss')
plt.xlabel('Epoch')
plt.legend(['Train', 'Validation'], loc='upper right')
plt.tight_layout()
plt.show()
```

Confusion Matrix for Classification


```

from sklearn.metrics import confusion_matrix, classification_report
import numpy as np
import seaborn as sns

y_pred = model.predict(X_test)
y_pred_classes = np.argmax(y_pred, axis=1) # For multi-class
# OR y_pred_classes = (y_pred > 0.5).astype(int) # For binary

cm = confusion_matrix(y_test, y_pred_classes)
plt.figure(figsize=(8, 6))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
plt.title('Confusion Matrix')
plt.ylabel('True Label')
plt.xlabel('Predicted Label')
plt.show()

# Detailed classification metrics
print(classification_report(y_test, y_pred_classes))

```

ROC Curve and AUC (for Binary Classification)

```

from sklearn.metrics import roc_curve, auc

# Get predicted probabilities for positive class
y_pred_prob = model.predict(X_test)[:, 1] # For multi-class with 2 classes
# OR y_pred_prob = model.predict(X_test) # For binary with sigmoid

# Calculate ROC curve
fpr, tpr, thresholds = roc_curve(y_test, y_pred_prob)
roc_auc = auc(fpr, tpr)

# Plot ROC curve
plt.figure(figsize=(8, 6))
plt.plot(fpr, tpr, label=f'AUC = {roc_auc:.3f}')
plt.plot([0, 1], [0, 1], 'k--') # Random performance line
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')

```

```
plt.title('ROC Curve')
plt.legend(loc='lower right')
plt.show()
```

Hyperparameter Tuning

Grid Search

```
from sklearn.model_selection import GridSearchCV
from tensorflow.keras.wrappers.scikit_learn import KerasClassifier

def create_model(units=64, dropout_rate=0.2, learning_rate=0.001):
    model = Sequential([
        Dense(units, input_shape=(input_dim,), activation='relu'),
        Dropout(dropout_rate),
        Dense(1, activation='sigmoid')
    ])
    model.compile(
        optimizer=Adam(learning_rate=learning_rate),
        loss='binary_crossentropy',
        metrics=['accuracy']
    )
    return model

# Create wrapper for Keras model
model = KerasClassifier(build_fn=create_model, epochs=20, batch_size=32, verbose=0)

# Define grid
param_grid = {
    'units': [32, 64, 128],
    'dropout_rate': [0.1, 0.3, 0.5],
    'learning_rate': [0.01, 0.001, 0.0001]
}

# Perform grid search
grid = GridSearchCV(estimator=model, param_grid=param_grid, cv=3)
grid_result = grid.fit(X_train, y_train)
```

```
# Best parameters
print(f"Best: {grid_result.best_score_:.4f} using {grid_result.best_params_}")
```

Random Search

```
from sklearn.model_selection import RandomizedSearchCV
from scipy.stats import randint, uniform

# Define parameter distribution
param_dist = {
    'units': randint(16, 256),
    'dropout_rate': uniform(0.1, 0.5),
    'learning_rate': uniform(0.0001, 0.01)
}

# Perform random search
random_search = RandomizedSearchCV(
    estimator=model,
    param_distributions=param_dist,
    n_iter=20, # Number of parameter settings sampled
    cv=3
)
random_search_result = random_search.fit(X_train, y_train)
```

Bayesian Optimization (with Optuna)

```
import optuna

def objective(trial):
    # Define hyperparameters to optimize
    units = trial.suggest_int('units', 16, 256)
    dropout_rate = trial.suggest_float('dropout_rate', 0.1, 0.5)
    learning_rate = trial.suggest_float('learning_rate', 1e-5, 1e-2, log=True)

    # Build and train model
```

```

model = Sequential([
    Dense(units, input_shape=(input_dim,), activation='relu'),
    Dropout(dropout_rate),
    Dense(1, activation='sigmoid')
])
model.compile(
    optimizer=Adam(learning_rate=learning_rate),
    loss='binary_crossentropy',
    metrics=['accuracy']
)

# Use early stopping to prevent overfitting
early_stopping = EarlyStopping(
    monitor='val_loss',
    patience=5,
    restore_best_weights=True
)

history = model.fit(
    X_train, y_train,
    validation_split=0.2,
    epochs=50,
    batch_size=32,
    callbacks=[early_stopping],
    verbose=0
)

# Return best validation accuracy
return max(history.history['val_accuracy'])

# Create and run study
study = optuna.create_study(direction='maximize')
study.optimize(objective, n_trials=50)

# Best parameters
print(f"Best parameters: {study.best_params}")
print(f"Best validation accuracy: {study.best_value:.4f}")

```

Model Saving and Loading

Saving Methods

```
# Save entire model (architecture + weights + optimizer state)
model.save('my_model.h5') # HDF5 format
model.save('my_model')    # SavedModel format

# Save just weights
model.save_weights('model_weights.h5')

# Save architecture only (as JSON)
model_json = model.to_json()
with open("model_architecture.json", "w") as json_file:
    json_file.write(model_json)
```

Loading Methods

```
from tensorflow.keras.models import load_model, model_from_json

# Load complete model
loaded_model = load_model('my_model.h5')
# OR
loaded_model = load_model('my_model')

# Load weights into existing model
model.load_weights('model_weights.h5')

# Load architecture and then weights separately
with open('model_architecture.json', 'r') as json_file:
    loaded_model_json = json_file.read()
model = model_from_json(loaded_model_json)
model.load_weights('model_weights.h5')
```

Callbacks and Training Control

Early Stopping

```

from tensorflow.keras.callbacks import EarlyStopping

early_stopping = EarlyStopping(
    monitor='val_loss', # or 'val_accuracy'
    patience=5,        # number of epochs with no improvement
    restore_best_weights=True,
    verbose=1
)

```

TensorBoard

```

from tensorflow.keras.callbacks import TensorBoard
import datetime

log_dir = "logs/fit/" + datetime.datetime.now().strftime("%Y%m%d-%H%M%S")
tensorboard_callback = TensorBoard(
    log_dir=log_dir,
    histogram_freq=1,
    write_graph=True
)

# View in terminal: tensorboard --logdir logs/fit

```

Model Checkpoint

```

from tensorflow.keras.callbacks import ModelCheckpoint

checkpoint = ModelCheckpoint(
    'model_epoch_{epoch:02d}_val_loss_{val_loss:.2f}.h5',
    monitor='val_loss',
    save_best_only=True,
    save_weights_only=False,
    mode='min',
    verbose=1
)

```

Custom Callback

```
from tensorflow.keras.callbacks import Callback

class CustomLogger(Callback):
    def on_epoch_end(self, epoch, logs=None):
        if epoch % 5 == 0:
            print(f"\nEpoch {epoch}: val_loss improved to {logs['val_loss']:.4f}")
```